

1 Introduction

For this assignment we've been tasked with implementing a C program called `file`. This program will serve as an API and takes a file path as an input argument and then displays the type of file associated with that given path. The text file types this API must discern are: ASCII, ISO-8859, and UTF-8. We also need to implement error handling in the API. This implementation is a simplified version of the C `file(1)` utility.

2 Implementation

The majority of the code in the main function is concerned with error handling. We ensure that the number of given arguments, `argc`, corresponds to calling the function on a single argument in the terminal, otherwise, we write an error message and exit with failure. To handle non-existing files or files we have no permission to read, we simply check if `file` equals `NULL`. In the case it is, we use the provided `print error()` function to write the error message in `errno` and exit with success. A thing to note with this check is that it also prevents the program from using the `NULL` file as arguments in other functions, thus avoiding referencing a null pointer which would lead to segmentation fault.

We have implemented a function `fileType()` that takes a file as an input. This function is called in main and is used to determine what encoding is used for the file. In this function, several "type checker" functions are called in an ordered sequence to determine the file type through the process of elimination.

First, we check whether the file is empty. This is done with the function `isEmpty()`. The function makes use of the fact that the `getc` function reads a single character from an input stream and returns `EOF` if it's at the end of the file. Thus the file is an empty file and we can then return 1, which results in the `filetype` function returning the enum `"EMPTY"`.

The functions `isASCII`, `isUTF` and `isISO` all use a while loop to read each character from a file, using the `fread()` function. The while loop iterates through every character in a file and stores it in a variable `c` and terminates once `fread()` returns 0 (which is when no objects were read, indicating that `EOF` has been reached).

When we check if the file is an ASCII file, we have eliminated the possibility of the file being empty due to the functions being called in an ordered sequence, such that we first check if the file is empty and then afterwards we check if it's ASCII. Thereafter, we check if it's UTF-8. We've chosen to check for UTF-8 before ISO because, the change of certain byte sequences appearing seems lower than certain character values in ISO appearing. For example the byte sequence `11001110 10111011` which corresponds to character values of 206 and 187; this sequence is both valid as an UTF-8 character (Ï) and as two ISO characters (ž), therefore depending on which order we check for ISO and UTF-8 encoding will determine which encoding our program determines the file to be.

`isASCII()` and `isISO()` iterate through every byte of a file and use a conditional statement to check the decimal representation of the byte. If the bytes,

stored in an unsigned char *c*, aren't within the range $6 \leq c \leq 15$ and $31 \leq c \leq 27$ and is not 27, then we can conclude that it can't be an ASCII file. *isISO*, which is called after *isUTF*, also has a nested conditional statement for the case where *c* could be in the range $159 \leq c \leq 255$, since these decimal representations are a part of ISO but not ASCII

```
int isUTF(FILE *file) {
    fseek(file, 0, SEEK_SET);
    unsigned char c;
    int checkFollowBytes = 0;
    while (fread(&c, sizeof(char), 1, file) == 1) {
        if (checkFollowBytes != 0) {
            if ((c >> 6) == 2) { //(10 xxxxxx)
                // valid follow byte
                checkFollowBytes--;
                continue;
            } else {
                return 0; // Not valid followbyte: Not UTF -8
            }
        } else if ((c >> 7) == 0) { //(0 xxxxxxx)
            continue;
        } else if ((c >> 5) == 6) { //(110 xxxxx)
            checkFollowBytes = 1;
            continue;
        }
    }
}
```

UTF-8 files may contain both ASCII and ISO characters, so *isUTF()* therefore we need to check the file encoding to determine if it's an UTF-8 file. UTF-8 files are encoded in a specific way, where characters are stored in up to 4 bytes and each subsequent byte for that character begins with specific pattern of bits. We can use bit masking to check if the bits follow the pattern for a UTF-8 character.

How many follow bytes we have to check depends on the pattern that the bits follow, since there is a different pattern for if the UTF character uses 1,2,3, or 4 bytes. In the case where the character is stored in one byte, the byte follows the pattern 0xxxxxxx and has no following bytes. In the case where the character is stored in 2 bytes, the first byte has the pattern 110xxxxx and the second byte (the following byte) has the pattern 10xxxxxx. In the case where the character is stored in 3 bytes the first character has the pattern 1110xxxx and has two following bytes with the same pattern. Finally, with a 4 byte character we have the leading byte with the pattern 11110xxx and it has 3 following bytes. We use right bit-shifting for our conditional statements since we need at most the leading 5 bits of a byte. For example, if we look at the pattern of following bytes, which is 10xxxxxx, we can see that we're only interested in the first 2 bits as the rest are arbitrary. If we right shift this byte by 6 we'll always get the byte 00000010 which happens to be equal to 2 in decimal. So, our conditional statement will check if:

$(c \gg 6 == 2)$

is true. If so, the following byte does follow the pattern. We use this same method for checking the leading bytes, depending on how many arbitrary bits we have in our byte.

We use a *int checkFollowBytes* variable to keep track of how many following bytes we have to check. This value is initialised once we've found our

leading byte and decremented each time we check for a following byte. when all characters have been checked we do a final check with a ternary operator, where we check that `int checkFollowBytes == 0`, this handles the case where the last character in a file match one of the UTF-8 designated bit-sequences that isn't 0xxxxxxx. In this case we expect a number of follow bytes, but since it's EOF the function would return true, this is avoided by checking that `int checkFollowBytes == 0` before returning 1 (true) and returning 0 (false) if `int checkFollowBytes != 0`.

Finally, if none of the checks `isEmpty`, `isASCII`, `isISO`, nor `isUTF` return true, then we conclude that the file is data and follows none of the patterns.

3 Testing

The testing in this assignment is done by comparing the behaviour of our implementation of file with the standard `file(1)` utility. We were provided a shell script `test.sh` that carries out this comparison by outputting the value of the standard file utility on a sample file into an expected text file. This is then compared to the actual output from our implementation. To add test cases, we can simply extend the script with more samples. Some of the different cases that we tested include files:

- being empty.
- containing ASCII characters.
- containing ISO characters
- containing ISO characters surrounded by ASCII characters.
- containing only the NULL character.
- adhering to ISO and UTF-8 encoding.
- not adhering to ASCII, ISO or UTF-8 encoding.
- being non-readable.
- containing certain byte sequences that adhere to the UTF encoding provided by the assignment text
- containing certain byte sequences that almost adhere to the UTF encoding provided by the assignment text, but not quite.

To run the script, simply execute `bash test.sh` in the terminal.

3.1 Test Coverage and Outcome

The majority of our test cases passed and aligned with the standard `file(1)` utility. We have demonstrated that our implementation of file is somewhat capable of identifying the various file types: Empty, ASCII, ISO, and UTF-8 through character encoding. We tried to test on standard cases as well as more rare edge

cases for instance certain byte sequences that almost adhered to UTF-8 encoding. We also “hand” tested the error handling of our program e.g. by calling `./file` with more/no arguments, non-existing and non-readable files. However, not all test cases passed. In the following section, we will discuss some of our testing limitations.

3.2 Testing Limitations

A test script was provided:

When running our test we observe certain tests don’t pass. By the specifications in the assignment text we are to classify all files that aren’t ASCII, ISO or UTF-8 as data. However the `file(1)` utility can identify several file types such as “Non-ISO extended-ASCII text”, this is one of the file types the `file(1)` utility identifies when given “data” files such as a file containing the bytes: `11110xxx10xxxxxx`. Per the assignment text: “Everything else is a data file”, this means that if provided a text file only containing the follow-byte `10xxxxxx`, the program should classify this file as data; our implementation does this as per the specifications, however the `file(1)` utility says “very short file (no magic)”, since “Everything else is a data file”, we say that the implementation behaves correctly even though our test fails.

Per the assignment text we are told to recognise a null char (`'\x0'`) as a valid UTF char because the byte sequence `0xxxxxxx` could be a null char, this would also include character values from 1-6, The `file(1)` utility however identifies character values 0-6 to be of type data. Therefore our implementation will identify a file containing “hello, `'\x0'`world” as UTF-8, whereas the `file(1)` utility will classify this as a data file.

Given the reasoning and explanation in this sub-section, even if some test is failing, we don’t conclude that our implementation is identifying files incorrectly, it’s capabilities is simply to limited to correctly identify the many file-types `file(1)` utility is able to, which is fine since we were tasked to “implement a basic variant of this tool in C”.

4 Limitations and Potential Problems

Our implementation of file does have some potential problems and limitations. Our file implementation is somewhat limited compared to the actual `file(1)` utility that is able to try to identify more file types. When our file program tries to guess a file type, it does so by attempting to eliminate it of being a particular encoding by seeing if it follows the rules of that encoding. In the case a file could be multiple file types, we have had to choose which takes priority.

5 Conclusion

For this assignment we were tasked with implementing a program file that can check which encoding a file has. As per the requirements we’ve implemented support for distinguishing between empty files and the text files types: ASCII,

ISO, and UTF-8. Through testing our implementation seems to function as intended, with an API for the program that includes error handling. However due to the limited functionality in our implementation we observe that some tests fail, but even though some tests fail, we conclude that our implementation functions correctly due to the assignment text asking us to "implement a basic variant of this (file) tool in C". Therefore our implementation end up not being able to identify as many file types file(1) and classifying "everything else as data". We recognize that our file implementation did the correct thing, which is; it didn't identify something that isn't ASCII, ISO-8559-1 or UTF-8 as one of those things. Furthermore we note that our implementation could end up incorrectly identifying UTF-8 files due to the encoding for UTF-8 given in the assignment being a simplification of the real encoding for UTF-8.